

TatHack '26 – Team AI'll Be Back – Development Minutes

Meeting & Project Progress Notes

Members

1. Muhammed Muhyudheen T, NITC, B250355ME, github : <https://github.com/muhyudheen>
2. Abel T Joseph, NITC, B25XXXXCS, github : <https://github.com/abeltjoseph2005-art>

AI model used along the project : **Claude Opus 5.5**

Problem Statement : SupplyChainer

Environments

- Local (VS Code, WSL terminal)
- Claude code - agentic work

Purpose of this doc

- This doc tracks as minutes, from 28/09/26 8 : 30PM to 30/09/26 7 PM
- Written Manually by Muhyudheen (Team Lead) and Abel T Joseph
- No AI writing here, only manual words.

28/09/26 : 8 30 PM to 10 PM

What we have done in Order:

1. Explored the problem statement and Repo ([ReadMe.md](#))
2. Explored the Architectures and main ideas used (NLP based Semantic Filtering, Gradient boosting model, Frontend stack, Routing Algorithm used, CARF etc)
3. File structure exploration and Libraries used
4. Cloned the repo into a private repo on Muhyudheen Github (collabed with Team)

5. Initiated Claude code, feed the problem statement into it
6. Commanded it to initiate local setup with UV (uv init), and install key dependencies

7.Initial Testing

So we tested different scenarios like suez canal block, red sea conflict, hormuz closures, along one route, reversed the route and 4 cases alone showed around 10+ blocks. Testing is not only done by frontend but also through FastAPI Swagger docs.

The bugs identified during Day 1 is in the repo named [Bug_Report.md](#)

*Bugs are found by manual testing, some of the edge test cases are generated by opus(We let the opus to do surface sweeping to find the surface level bugs, which is many, many are dead variables, dead instances etc), the bug report was created by opus after feeding it the results from both API testing and frontend testing and after going through [main.py](#) and some other files in engine module manually.

We speedup the bug tracing at last as time is not being nice, so yeah, those bugs were found by opus, the ML core files like threat_[intelligence.py](#), real_dataset_builder.py(code module), Execution files (risk_model.pkl, label_encoders.pkl, nlp_anchors.pt, calibration_profiles.json) these are reviewed manually good as AI / ML is team's core strength.

Comment on "Real Dataset" (ref : code/real_dataset_builder)

Why the data is not real:

- Delays are gamma generated (Gamma distribution(alpha, beta))
- Every category is a coin flip. Transport mode (line 61), origin and destination (64–85), and weather (87) are picked uniformly at random. Each mode is about 25% of rows and each weather type about 33%, not real frequencies.
- Weather has no effect
- Road rows aren't real places
- Etc

12:47 am - Detailed bug report is to be generated and scheduled for tomorrow(29/09/26) morning

29/09/26 - 8 : 00 am Continuing

Created the bug_report.md after reviewing a few more bugs and made some decisions on how to approach the solutions. NPL Threat engine is never producing any threat (a critical bug). There is a clear data leakage. P85 model isn't really predicting p85, high chance it will trip on real data.

12 45 PM - Bug Fixing Round 1 - NLP Engine

How the fixing is done: we create a test script for each rounds (created by claude code), one by one fixing a bug, checking if it is implemented correctly, committing it (committed by calude code)

Bugs fixed

1. Added `map_location = 'cpu'` to threat_intelligence.py:157 - The cause of Threat being zero everywhere, earlier only GPU can run this
2. Fixed NLP loading failure,the warm-up in our report failure(backend/engine/threat_intelligence.py)
3. Inverted the noise threshold logic (switched `>=` to `<`) - earlier actual risks evaluate to zero while baseline noise bypasses it, causing negative margins to yield negative risk values (backend/engine/threat_intelligence.py)
4. Margin is based on chunks. One calm chunk can cancel out one alarming chunk and no threat at all. Now the margin is computed per chunk. (backend/engine/threat_intelligence.py)
5. CARF dropped sea threats *because* they were about the sea, broke on punctuation, and ignored rail and road; it now drops a threat only when the news is about other modes, and checks all four. (backend/engine/threat_intelligence.py)
6. Every edge was scored against an invented "congestion" sentence, creating fake threats; edges with no live news now get threat 0. (backend/engine/news_ingestion.py)

6 PM - Round 2 - Graph

1. There were so many connections with no reverse direction along the edges of the graph, now work in both directions, so ships and trains can make the return trip.
2. Bug was sea routes crossing through lands, and chokepoints is primarily deactivate and also 4 out of 5 choke points have exit but no enter (no port lists a line going through it but lists lines coming out of it) - fixed : Sea routes stay within ocean basins, pass through the real chokepoints (Malacca, Bab-el-Mandeb, Suez, Hormuz), and measure lane distance around land instead of straight across it.
3. The bug: The road auto-wire links every road-capable pair of hubs under 200 km apart, in both directions, ignoring water and borders. Fix : No more trucks driving across the

sea between countries separated by water (UK–France, Morocco–Spain, Indonesia–Singapore).

4. No self-loops or zero-length legs; hubs at the same spot get a 5 km minimum leg.
5. Trips start and end in whichever mode fits best, so port-to-port sea trips no longer pay truck handoffs.

9 PM - Round 3 - Scenarios - Majorly done by Claude code

1. Bug : The closed chokepoints is still letting the cargo through. Routes sail through a closed canal whenever the detour is longer. Fix : closed means closed, now those choke points are impassable.
2. Bug : Example : Chennai Flood is affecting the transport between PORT_SINGAPORE and PORT_CHENNAI with sea mode for no reason. The router is delaying any link that enters an affected hub. Fix : now a scenario only affects its own mode.
3. Minor fixes like mismatches between text and said values. The Dubai congestion scenario now applies the 48-hour delay its description announces (it used 24).
4. Request fields in fastapi Model declarations are free strings with no allowed-values check. An unknown scenario is playing as a 'No Scenario'. Fix : A misspelled scenario name now returns an error instead of silently running with no disruption.

11 PM - Round 4 - Audit maths - Completely done by Opus

1. Bug : The scenario delay is counted twice in the audit trace. Fix : the transit and transfer buckets get the base hours, and only the scenario bucket gets the delay.
2. Bug: The 10% Scenario surcharge never reaches total_cost.
3. "Baseline" risk is recorded after the scenario has already been applied, fix : Baseline risk is taken before scenario
4. Impossible saving percentages like 387% (Dashboard 1), 391% (Red Sea / Hormuz), 396% (API Suez), 11225% (Reverse). Each is always 15% of the route's own cost, Reduced risk exposure by 95%, that's just 100% minus the 5% default threat. The explanation numbers are formulas on the route's own values, not comparisons: 20% of transit hours, transfer hours ÷ 4, (1 – threat) × 100, and 15% of cost. Fix : Real comparison instead of invented formulas, example : "Fastest route: 617.1h, \$2,923, peak threat 0.85, 0 transfers. 87.6h quicker than SAFEST."
5. When two personas pick the same stops, the later card is dropped without a note. You won't see FASTEST, BALANCED, and SAFEST along, Fix : shared routes are named, not dropped

30 / 09 / 26 - 8 30 AM to 11 AM

Round 6 - Dataset and Model Training

We cleared the bugs the dataset code and also trained the model leakage free, we added 3 new files ``backend/ml/features.py``, ``backend/ml/dataset.py``, ``backend/ml/train.py``,

- [features.py](#) : It have one function, `leg_features()`, given a leg, it returns the features of it
- [dataset.py](#) : Builds the 50k training rows: picks real legs from our graph, calls `leg_features()` on each, then simulates a delay for each row.
- [train.py](#) : trains the p50/p85/p95 models on those rows and reports how accurate they are.

Bugs fixed along the way

1. The data was simulated but labelled "Real Historical". The new builder says in its header that the data is simulated, and marks every invented number as "assumed".
2. The gamma used only the median with a fixed shape, but originally it said median and p90, so the p90s were far off (Suez 144h came out as 28h). Now the shape and scale are fitted so both median and p90 match, within 10%
3. The news score was computed from the row's own delay, so the model learned the answer from its input. Now news is drawn before the delay.
4. Weather was drawn but never used (sea at storm didn't increase the delay). Now weather raises the en-route delay: stormy sea p85 is 1.56× clear.
5. Delay ignored distance, so Shanghai→Singapore had the same delay as Shanghai→Rotterdam. Delay depended on original Anchors. Now en-route delay is a share of the leg's travel time, so long legs have higher delay.
6. Every category is drawn uniformly at random like 25% per mode, 33 % per weather, Suez canal as an origin port, Houston port as a rail hub. Now rows are real legs of our route graph, balanced only by mode.
7. Some events are deadlogic. Only 191 out of 50k rows got one incident, because 5% draw only applied when mode and location matches, example Red Sea Security 2024 was drawn 457 times, applied 0 times : its Location, Dubai Logistics hub is an air hub not a sea port.

Round 7 11 AM to 2 PM - Model

Before Round 7 : The router never called the model. No ML was happening there. And also the trained model in the Execution files were built on the fake rows from the `real_dataset_builder.py`. We changed the dataset, fixed the bugs, and generated a new one. Trained a new model on them and connected it to the Graph. Connection was

mainly done by Claude Code. Also we added New Weather API, Open Meteo (No API Key is needed), because weather is dead in the original data.

Right now the model is predicting `delay\_h` hours of delay beyond schedule time. Its built from three parts: 1. Dwell (waiting at the hub where cargo is handled), its applied only at the end of the leg. We used organizers' own figures (The median and P90 per location), 2. A share of the leg's Scheduled time like 5%, or 15% on a bad day(p95). This is our assumption also weather ($1 + k \cdot \text{weather}$) where $k = 1.0$ for sea, 0.8 for air, 0.5 for road and 0.3 for rail. 3. Incidents(disasters): different for different types of legs. For a normal leg, its 4%, legs touching an open strait, cape or sea route - 8%, canal legs - 4%, bcoz canals 144h p90 already contains the 2021 Suez canal blockage so it will be doubling.

We removed the leakage, originally news_severity is calculated from the delay, now we removed that, injected weather, whether there is an incident happening or not etc.. only then delay is computed.

Weather frequencies are assumed and are real WMO Weather codes on training and they're turned into 0-1 using the same table the live Open Meteo provider uses, so training and app speak the same scale.

We trained three models: p50(the typical delay), p85(a bad day 85% trips are below it), and p95 a very bad day. The old code only trained p85, so no range was possible.

How a model learns p85: for this, we want a line with 85% of outcomes below it, so the penalty is lopsided. If the real delay is above our guess, each hour costs 0.85 and if its below each hour costs only 0.15 So the model leans to guess high enough that its under only about 15% of the time.

Model Type : we use `HistGradientBoostingRegressor()` from skicit-learn, with the quantile loss.

Categories are encoded using OHE(one hot encoder).

We didn't use RandomSplit(leakage risk), we grouped the rows by hub pair(both directions and all modes count as one route). Then 20% of routes go to the test set entirely.

We checked the performance on unseen data and these were the two mode's results:

Sea : p50: 49.3%, p85: 84.4%, p95: 94.7%

Air : p50: 51.3%, p85: 84.7%, p95: 94.5%

Calibrated : every quantile is within 2.2% of its target. So 85% means 85% for the new model.

3 PM to 4 PM

Cargo Mode: Goal is Make the **cargo type** and **priority** inputs actually do something.

There is cargo type and priority multiply unused. So we are implementing it to the router graph, removing the point which violates the condition for cargo, and giving a generic error message.

How :

1. Before routing, each persona's graph drops the edges of every mode that can't carry the cargo. That's the same mechanism that already removes closed hubs, so it's small and easy to explain.
2. Priority changes how much BALANCED cares about time. Its time weight becomes $0.3 \div \text{multiplier}$: urgent ≈ 0.43 (time matters more), low = 0.25. FASTEST (time only) and SAFEST (risk first) stay as they are.
3. The response says what applied: e.g. `cargo_rules: {cargo_type, excluded_modes, priority, balanced_time_weight}`
4. **Clear errors:**
 - a. an unknown cargo type or priority $\rightarrow$ 400 listing the valid values
 - b. a conflict, e.g. perishable cargo with transport preference "sea" $\rightarrow$ 400 "perishable_urgent cargo can't go by sea"
 - c. no route left $\rightarrow$ 404 naming the rule

Dashboard now has cargo and priority inputs.

Final Stage

1. Claude managed the frontend, edited dashboard, new cargo type and priority settings, put the delay model on screen(ETA ranges, SHAP comparisons)
2. Updated the [README.md](#) with new fixes and fields
3. Testing
4. Video
5. Final Documentation

Additional Feature Enhancements Done

1. New delay model (p50/p85/p95) wired into routing
2. ETA ranges on cards
3. SHAP delay drivers
4. Live Open-Meteo weather
5. Cargo type and priority controls with reasons

6. Real model status in [/api/status](#)
7. Sea network with basins, gates and lanes
8. 0 → 133 backend + 30 frontend tests
9. uv setup
10. Bug docs and the AI-usage log

THANK YOU